



Enter keyword o...



Additional Content In: [Japanese](#) [German](#) [Hebrew](#) [Korean](#) [Chinese](#) [Spanish](#)

Debian: Getting Started with the BeagleBone Black

[RobertCNelson](#) 1 March 24, 2021, 9:28pm

This is a page about TI's Cortex-A8 based; BeagleBone Black.

Availability

Boards:

- [BeagleBone Black](#) at Digi-Key
- [BeagleBone Black](#) at Digi-Key
- [BeagleBone Black Industrial](#) at Digi-Key
- [BeagleBone Black Wireless](#) at Digi-Key
- [BeagleBone Green](#) at Digi-Key
- [BeagleBone Green Wireless](#) at Digi-Key
- [BeagleBone Green Gateway](#) at Digi-Key

Power Supplies:

- [USB Micro for BeagleBone Green](#) at Digi-Key

Cables:

- [\(USB to serial adapter\) TTL-232R-3V3](#) at Digi-Key

Basic Requirements

- Running a recent supported release of Debian, Fedora or Ubuntu on a x86 64bit based PC; without OS Virtualization Software.
- Many of the listed commands assume /bin/bash as the default shell.
- [Kernel.org](#) Cross Compilers - <https://mirrors.edge.kernel.org/pub/tools/crosstool/>
- Bootloader
 - Das U-Boot - the Universal Boot Loader: <http://www.denx.de/wiki/U-Boot>
 - Source: <https://github.com/u-boot/u-boot/>
- Linux Kernel
 - Linus's Mainline tree: <https://git.kernel.org/pub/scm/linux/kernel/git/>

torvalds/linux.git

- ARM based rootfs
- Debian: <https://www.debian.org>

Learning

- Fun with Beagle – Exploring the GPU: <https://university.imgtec.com/fun-with-beagle-gpu/>
- Fun with Beagle – Exploring the GPU Video: <https://university.imgtec.com/fun-with-beagle-video/>

ARM Cross Compiler: GCC

This is a pre-built (64bit) version of GCC that runs on generic linux, sorry (32bit) x86 users, it's time to upgrade...

Download/Extract:

```
#user@localhost:~$  
wget -c https://mirrors.edge.kernel.org/pub/tools/crosstool/files/bin/x86_64  
tar -xvf x86_64-gcc-11.3.0-nolibc-arm-linux-gnueabi.tar.xz  
export CC=`pwd`/gcc-11.3.0-nolibc/arm-linux-gnueabi/bin/arm-linux-gnueabi-
```

Test Cross Compiler:

```
#user@localhost:~$  
${CC}gcc --version
```

```
#Test Output:  
arm-linux-gnueabi-gcc (GCC) 11.3.0  
Copyright (C) 2021 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Bootloader: U-Boot

Das U-Boot – the Universal Boot Loader: <http://www.denx.de/wiki/U-Boot>

Depending on your Linux Distribution, you will also need a host gcc and other tools, so with Debian/Ubuntu start with installing the build-essential meta package.

```
#Requirements  
sudo apt install bison build-essential flex swig
```

Download:

```
#user@localhost:~$  
git clone -b v2022.04 https://github.com/u-boot/u-boot --depth=1  
cd u-boot/
```

Patches:

```
#user@localhost:~/u-boot$  
git pull --no-edit https://git.beagleboard.org/beagleboard/u-boot.git v2022.04
```

Configure and Build:

```
#user@localhost:~/u-boot$  
make ARCH=arm CROSS_COMPILE=${CC} distclean  
make ARCH=arm CROSS_COMPILE=${CC} am335x_evm_defconfig  
make ARCH=arm CROSS_COMPILE=${CC}
```

Linux Kernel

This script will build the kernel, modules, device tree binaries and copy them to the deploy directory.

Mainline

Download:

```
#~/  
git clone https://github.com/RobertCNelson/bb-kernel ./kernelbuildscripts  
cd kernelbuildscripts/
```

For am33x-v5.4 (Longterm 5.4.x):

```
#user@localhost:~/kernelbuildscripts$  
git checkout origin/am33x-v5.4 -b tmp
```

For am33x-rt-v5.4 (Longterm 5.4.x + Real-Time Linux):

```
#user@localhost:~/kernelbuildscripts$  
git checkout origin/am33x-rt-v5.4 -b tmp
```

For am33x-v5.10 (Longterm 5.10.x):

```
#user@localhost:~/kernelbuildscripts$  
git checkout origin/am33x-v5.10 -b tmp
```

For am33x-rt-v5.10 (Longterm 5.10.x + Real-Time Linux):

```
#user@localhost:~/kernelbuildscripts$  
git checkout origin/am33x-rt-v5.10 -b tmp
```

For am33x-v5.15 (Longterm 5.15.x):

```
#user@localhost:~/kernelbuildscripts$  
git checkout origin/am33x-v5.15 -b tmp
```

For am33x-rt-v5.15 (Longterm 5.15.x + Real-Time Linux):

```
#user@localhost:~/kernelbuildscripts$  
git checkout origin/am33x-rt-v5.15 -b tmp
```

Build:

```
#user@localhost:~/kernelbuildscripts$  
./build_kernel.sh
```

TI BSP

Download:

```
#~/  
git clone https://github.com/RobertCNelson/ti-linux-kernel-dev ./kernelbuild  
cd kernelbuildscripts/
```

For TI v4.19.x:

```
#~/kernelbuildscripts/  
git checkout origin/ti-linux-4.19.y -b tmp
```

For TI v4.19.x: Real-Time

```
#~/kernelbuildscripts/  
git checkout origin/ti-linux-rt-4.19.y -b tmp
```

For TI v5.4.x:

```
#~/kernelbuildscripts/  
git checkout origin/ti-linux-5.4.y -b tmp
```

For TI v5.4.x: Real-Time

```
#~/kernelbuildscripts/
```

```
git checkout origin/ti-linux-rt-5.4.y -b tmp
```

For TI v5.10.x:

```
#~/kernelbuildscripts/  
git checkout origin/ti-linux-5.10.y -b tmp
```

For TI v5.10.x: Real-Time

```
#~/kernelbuildscripts/  
git checkout origin/ti-linux-rt-5.10.y -b tmp
```

Build:

```
#user@localhost:~/kernelbuildscripts$  
./build_kernel.sh
```

Root File System

Debian 11

User	Password
debian	temppwd
root	root

Download:

```
#user@localhost:~$  
wget -c https://rcn-ee.com/rootfs/eewiki/minfs/debian-11.5-minimal-armhf-2022-10-06.tar.xz
```

Verify:

```
#user@localhost:~$  
sha256sum debian-11.5-minimal-armhf-2022-10-06.tar.xz
```

```
#sha256sum output:  
0ea53b23483af4bcccca53f3fd907b5e404f2f607a6577d78e8c4daf5b869ad0  debian-11.5-minimal-armhf-2022-10-06.tar.xz
```

Extract:

```
#user@localhost:~$  
tar xf debian-11.5-minimal-armhf-2022-10-06.tar.xz
```

Setup microSD card

We need to access the External Drive to be utilized by the target device. Run `lsblk` to help figure out what linux device has been reserved for your External Drive.

```
#Example: for DISK=/dev/sdX
lsblk
NAME      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
sda         8:0    0 465.8G  0 disk
├─sda1       8:1    0   512M  0 part /boot/efi
└─sda2       8:2    0 465.3G  0 part /
sdb         8:16    1   962M  0 disk
└─sdb1       8:17    1   961M  0 part
```

<- Development Machine Root
<- microSD/USB Storage Device
<- microSD/USB Storage Partition

```
#Thus you would use:
export DISK=/dev/sdb
```

```
#Example: for DISK=/dev/mmcblkX
lsblk
NAME      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
sda         8:0    0 465.8G  0 disk
├─sda1       8:1    0   512M  0 part /boot/efi
└─sda2       8:2    0 465.3G  0 part /
mmcblk0    179:0    0   962M  0 disk
└─mmcblk0p1 179:1    0   961M  0 part
```

<- Development Machine Root
<- microSD/USB Storage Device
<- microSD/USB Storage Partition

```
#Thus you would use:
export DISK=/dev/mmcblk0
```

Erase partition table/labels on microSD card:

```
sudo dd if=/dev/zero of=${DISK} bs=1M count=10
```

Install Bootloader:

```
#user@localhost:~$
sudo dd if=./u-boot/MLO of=${DISK} count=2 seek=1 bs=128k
sudo dd if=./u-boot/u-boot-dtb.img of=${DISK} count=4 seek=1 bs=384k
```

Create Partition Layout:

With util-linux v2.26, `sfdisk` was rewritten and is now based on `libfdisk`.

```
#Check the version of sfdisk installed on your pc is atleast 2.26.x or newer
sudo sfdisk --version
```

```
#Example Output
sfdisk from util-linux 2.27.1
```

```
#sfdisk >= 2.26.x
sudo sfdisk ${DISK} <<-__EOF__
4M,,L,*
__EOF__
```

Format Partition:

With mkfs.ext4 1.43, we need to make sure metadata_csum and 64bit ext4 features are disabled.

As the version of U-Boot needed for this target CAN NOT correctly handle reading files with these newer ext4 options.

```
#mkfs.ext4 -V
sudo mkfs.ext4 -V
mke2fs 1.43-WIP (15-Mar-2016)
    Using EXT2FS Library version 1.43-WIP
```

```
#mkfs.ext4 >= 1.43
for: DISK=/dev/mmcblkX
sudo mkfs.ext4 -L rootfs -O ^metadata_csum,^64bit ${DISK}p1

for: DISK=/dev/sdX
sudo mkfs.ext4 -L rootfs -O ^metadata_csum,^64bit ${DISK}1
```

Mount Partition:

On most systems these partitions may be auto-mounted...

```
sudo mkdir -p /media/rootfs/

for: DISK=/dev/mmcblkX
sudo mount ${DISK}p1 /media/rootfs/

for: DISK=/dev/sdX
sudo mount ${DISK}1 /media/rootfs/
```

Backup Bootloader

This version of MLO/u-boot.img will be used on the “eMMC” flasher script on this page.

```
#~/
sudo mkdir -p /media/rootfs/opt/backup/uboot/
sudo cp -v ./u-boot/MLO /media/rootfs/opt/backup/uboot/
sudo cp -v ./u-boot/u-boot-dtb.img /media/rootfs/opt/backup/uboot/
```

Install Kernel and Root File System

To help new users, since the kernel version can change on a daily basis. The kernel building scripts listed on this page will now give you a hint of what kernel version was built.

```
-----
Script Complete
eeewiki.net: [user@localhost:~$ export kernel_version=5.X.Y-Z]
-----
```

Copy and paste that “export kernel_version=5.X.Y-Z” exactly as shown in your own build/desktop environment and hit enter to create an environment variable to be used later.

```
export kernel_version=5.X.Y-Z
```

Copy Root File System

```
#Debian; Root File System: user@localhost:~$
sudo tar xfvp ./debian-***-armhf-*/armhf-rootfs-*.tar -C /media/rootfs/
sync
```

Set uname_r in /boot/uEnv.txt

```
#user@localhost:~$
sudo sh -c "echo 'uname_r=${kernel_version}' >> /media/rootfs/boot/uEnv.txt"
```

Copy Kernel Image

Kernel Image:

```
#user@localhost:~$
sudo cp -v ./kernelbuildscripts/deploy/${kernel_version}.zImage /media/rootf
```

Copy Kernel Device Tree Binaries

```
#user@localhost:~$
sudo mkdir -p /media/rootfs/boot/dtbs/${kernel_version}/
sudo tar xfv ./kernelbuildscripts/deploy/${kernel_version}-dtbs.tar.gz -C /m
```

Copy Kernel Modules


```
#user@localhost:~$
sudo tar xfv ./kernelbuildscripts/deploy/${kernel_version}-modules.tar.gz -C
```

File Systems Table (/etc/fstab)

```
#user@localhost:~/ $
sudo sh -c "echo '/dev/mmcblk0p1 / auto errors=remount-ro 0 1' >> /media
```

Networking

Edit: /etc/network/interfaces

```
sudo nano /media/rootfs/etc/network/interfaces
```

Add:

```
#/etc/network/interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
```

Remove microSD/SD card

```
sync
sudo umount /media/rootfs
```

HDMI

This sections assumes you have already installed your favorite xorg based window manager, such as lxde, xfce, kde, gnome, etc... These are packages that need to be installed on top of your selected windows manager and an xorg.conf needed to correctly setup the video interface.

*Note: If the cursor doesn't show up right away, first hit: **ctrl-alt-f1** then: **ctrl-alt-f7** after which it 'should' show up...*

Make sure to install, fbdev driver and xrandr utilities:

```
sudo apt update
sudo apt install read-edid xserver-xorg-video-fbdev x11-xserver-utils
```

```
#/etc/X11/xorg.conf
Section "Monitor"
    Identifier          "Builtin Default Monitor"
EndSection
```

```

Section "Device"
    Identifier      "Builtin Default fbdev Device 0"
    Driver          "fbdev"
EndSection
Section "Screen"
    Identifier      "Builtin Default fbdev Screen 0"
    Device          "Builtin Default fbdev Device 0"
    Monitor         "Builtin Default Monitor"
EndSection
Section "ServerLayout"
    Identifier      "Builtin Default Layout"
    Screen          "Builtin Default fbdev Screen 0"
EndSection

```

xrandr:

```

xrandr
xrandr --output HDMI-0 --mode 1024x768 --rate 60

```

xrandr (over serial/ssh)

```

xrandr -display :0.0 -q
xrandr -display :0.0 --output HDMI-0 --mode 1024x768 --rate 60

```

eMMC

Script to copy your microSD card to eMMC: (this will need these packages installed: initramfs-tools dosfstools rsync)

```

wget https://raw.githubusercontent.com/RobertCNelson/boot-scripts/master/tools/bbb-eMMC-flasher-eewiki-ext4.sh
chmod +x bbb-eMMC-flasher-eewiki-ext4.sh
sudo /bin/bash ./bbb-eMMC-flasher-eewiki-ext4.sh

```

U-Boot Overlays

Full Documentation: [readme|[Beagleboard:BeagleBoneBlack Debian - eLinux.org](#)]

Any issues:

```

#Any issues run:
sudo /opt/scripts/tools/version.sh

```

Enable:

```

#/boot/uEnv.txt
enable_uboot_overlays=1

```

To Disable: eMMC:

```
#!/boot/uEnv.txt
disable_uboot_overlay_emmc=1
```

To Disable: HDMI VIDEO & AUDIO:

```
#!/boot/uEnv.txt
disable_uboot_overlay_video=1
```

To Disable: HDMI AUDIO:

```
#!/boot/uEnv.txt
disable_uboot_overlay_audio=1
```

To Disable: WL1835:

```
#!/boot/uEnv.txt
disable_uboot_overlay_wireless=1
```

To Disable: BB-ADC:

```
#!/boot/uEnv.txt
disable_uboot_overlay_adc=1
```

U-Boot: override detected capes

```
#!/boot/uEnv.txt
uboot_overlay_addr0=<file0>.dtbo
uboot_overlay_addr1=<file1>.dtbo
uboot_overlay_addr2=<file2>.dtbo
uboot_overlay_addr3=<file3>.dtbo
```

U-Boot: disable auto-loading of detected capes

```
#!/boot/uEnv.txt
disable_uboot_overlay_addr0=1
disable_uboot_overlay_addr1=1
disable_uboot_overlay_addr2=1
disable_uboot_overlay_addr3=1
```

U-Boot: load 4 more un-detected capes

```
#!/boot/uEnv.txt
uboot_overlay_addr4=<file4>.dtbo
uboot_overlay_addr5=<file5>.dtbo
uboot_overlay_addr6=<file6>.dtbo
```

```
uboot_overlay_addr7=<file7>.dtbo
```

U-Boot: PRU Options (v4.19.x-ti)

```
#/boot/uEnv.txt  
uboot_overlay_pru=AM335X-PRU-RPROC-4-19-TI-00A0.dtbo
```

U-Boot: PRU Options

```
#/boot/uEnv.txt  
uboot_overlay_pru=AM335X-PRU-UI0-00A0.dtbo
```

U-Boot: Cape Universal

```
#/boot/uEnv.txt  
enable_uboot_cape_universal=1
```

Comments

Any questions or comments please go to our TechForum: [TechForum](#)

[About "Debian: Getting Started with the BeagleBone Black"](#)

[BeagleBone Black boot fail](#)

[Issue booting 5.10.47-bone-rt-r44 kernel on the BeagleBoneBlack](#)

[AM3358: eMMC flasher is not working on a custom board based on beaglebone black](#)

[AM3358 : eMMC is not detect in Linux on a custom board based on beaglebone black](#)

[Debian: BeagleBone Black入門ガイド](#)

[Can't download root filesystem for BeagleBone Black](#)

[BBG: BB-AAIM-LCD ubuntu 20.04 and 5.10.x-bone-rt](#)

[Board_ID, U-BOOT related question](#)

[BeagleBone Black USB port Boot](#)

[Ethernet device is not detecting on ubuntu 20.04 LTS on BBG](#)

[Ubuntu Server 20.04](#)

[데비안: 비글본 블랙 시작하기](#)

[STM32MP157C-DK2 HDMI Output of Xorg](#)

[Building and Trying...BeagleBone Black and Aufs](#)

[Beaglebone black usb-audio broken](#)

[BBB usb gadget questions](#)

[Kernel and U-boot overlay for a custom Board based on Beaglebone Black](#)

[Usb0 now showing beaglebone black](#)

[Buildroot on Beaglebone?](#)

[About window on Beaglebone](#)

[BeagleBone Black - microSD issues](#)

[AM3358 : eMMC is not detect in Linux on a custom board based on beaglebone black](#)

[BBG: BB-AAIM-LCD ubuntu 20.04 and 5.10.x-bone-rt](#)

[Shell access to the beaglebone black](#)

[About "Debian: Getting Started with the BeagleBone Black"](#)

[Timed out waiting for device dev-ttyS0.device](#)

[Boot failure after running bbb-eMMC-flasher-eewiki-ext4.sh](#)

[About the datetime and soft power off for BBB](#)

[Unable to login to the BeagleBone Black](#)

[Shell access to the beaglebone black](#)

[Shell access to the beaglebone black](#)

[Shell access to the beaglebone black](#)

[\[BBB\] Kernel build error in Ubuntu 24.04 LTS](#)

[Unable to login to the Beaglebone Black with custom image](#)

[Ethernet device is not detecting on ubuntu 20.04 LTS on BBG](#)

[RobertCNelson](#) Split this topic 4 October 15, 2023, 4:42pm

A post was split to a new topic: [Automating Kernel Build scripts](#)

[RobertCNelson](#) Split this topic 5 February 26, 2024, 2:45pm

10 posts were split to a new topic: [BeagleBone Black - microSD issues](#)

Related topics

Topic	Replies	Activity
Debian: Getting Started with the BeagleBone Blue	0	March 24, 2021
Debian: Getting Started with the BeagleBoard-X15	0	August 14, 2017
Debian: Getting Started with the BeagleBone-AI	2	December 22, 2021
Debian: Getting Started with the SeeedStudio BeagleBone Green Gateway	0	March 24, 2021
Debian: Getting Started with the BeagleBone seeed-technology , evaluation-boards , engineering-briefs	0	March 24, 2021